

Minta ZH felkészülési jegyzet .NET MAUI-hoz

Ez a jegyzet a `MintaZH_25261.pdf` feladat és a `MAUIMintaZH` tanári megoldás alapján készült. A cél az, hogy éles ZH-n ne vakon kezdj neki, hanem legyen egy bevált sorrended.

A feladat lényege

Egy egyszerű bevásárlólista alkalmazást kell készíteni .NET MAUI-ban.

A megoldandó részek:

- adatmodell: név, leírás, mennyiség, kép, megvásárolva-e
- főoldali lista `CollectionView`-val
- új elem hozzáadása külön oldalon
- kiválasztott elem törlése
- termékadat megosztása
- `MainPage` gombstílus: `MidnightBlue` háttér, félkövér szöveg, 5 margin
- `About` oldal `Flyout` menüből
- lista mentése és betöltése `AppDataDirectory`-ból

Pontozás szerinti súly

Erre érdemes figyelni, mert ZH-n nem mindig a legszebb megoldás adja a legtöbb pontot.

- Modell: 5 pont
- Főoldali lista kinézet: 25 pont
- Új oldalra navigálás és törlés: 15 pont
- Új elem felvitele és visszaadása: 25 pont
- Megosztás: 5 pont
- Gombstílus: 5 pont
- `About` oldal `Flyout`-ból: 10 pont
- Mentés/betöltés fájlba: 10 pont

Vagyis a legfontosabb: működjön a lista, az új elem hozzáadása, a navigáció és a mentés.

Ajánlott megoldási sorrend ZH-n

1. Projekt létrehozása

Indíts egy új .NET MAUI App projektet.

Ha lehet, először Windows célplatformon tesztelj, mert gyorsabb. Ha a ZH Androidot kér, a végén Androidon is próbáld.

2. NuGet csomag

Kell a `CommunityToolkit.Mvvm`:

```
<PackageReference Include="CommunityToolkit.Mvvm" Version="8.2.2" />
```

Újabb projektben lehet más verzió is, például `8.4.0`, de a lényeg, hogy elérhető legyen:

```
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;
```

3. Modell osztály

Hozz létre egy `ShoppingListItem.cs` fájlt.

Minimum mezők:

```

using CommunityToolkit.Mvvm.ComponentModel;

namespace MAUIMintaZH;

public partial class ShoppingListItem : ObservableObject
{
    [ObservableProperty]
    string name;

    [ObservableProperty]
    string description;

    [ObservableProperty]
    int quantity;

    [ObservableProperty]
    string imageURL;

    [ObservableProperty]
    bool isDone;
}

```

Fontos: az [ObservableProperty] miatt a name mezőből automatikusan Name property lesz. Ezt használja majd a XAML binding.

4. MainPageViewModel

Ez tartalmazza a lista adatait, a kiválasztott elemet, a commandokat és a mentés/betöltést.

Szükséges elemek:

- ObservableCollection<ShoppingListItem> ShoppingList
- [ObservableProperty] ShoppingListItem selectedItem
- AddNewCommand
- DeleteSelectedCommand
- ShareSelectedCommand
- LoadFromJson
- SaveToJson
- [QueryProperty(nameof(NewItem), "newitem")]

A tanári megoldásban az új elem visszaadása így történik:

```

[QueryProperty(nameof(NewItem), "newitem")]
public partial class MainPageViewModel : ObservableObject
{
    public ObservableCollection<ShoppingListItem> ShoppingList { get; private set; }

    public ShoppingListItem NewItem
    {
        set { ShoppingList.Add(value); }
    }
}

```

Ez azt jelenti, hogy amikor az új elem oldal visszanavigál a főoldalra és átadja a newItem paramétert, akkor a NewItem setter lefut, és hozzáadja az elemet a listához.

5. Főoldal XAML

A főoldal lényege:

- CollectionView
- ItemsSource="{Binding ShoppingList}"
- SelectedItem="{Binding SelectedItem}"
- SelectionMode="Single"
- DataTemplate
- CheckBox, Image, Label elemek
- alul gombok

Váz:

```

<Grid RowDefinitions="*,Auto">
    <CollectionView ItemsSource="{Binding ShoppingList}"
        SelectedItem="{Binding SelectedItem}"
        SelectionMode="Single">
        <CollectionView.ItemTemplate>
            <DataTemplate x:DataType="local:ShoppingListItem">
                <Border Stroke="Black" StrokeThickness="1" Margin="5">
                    <Grid ColumnDefinitions="*,*,3*">
                        <CheckBox IsChecked="{Binding IsDone}" />
                        <Image Source="{Binding ImageURL}" Grid.Column="1" HeightRequest="100" />
                        <VerticalStackLayout Grid.Column="2">
                            <Label Text="{Binding Name}" FontAttributes="Bold" />
                            <Label Text="{Binding Description}" />
                            <Label Text="{Binding Quantity, StringFormat='Quantity: {0}'}" />
                        </VerticalStackLayout>
                    </Grid>
                </Border>
            </DataTemplate>
        </CollectionView.ItemTemplate>
    </CollectionView>

    <HorizontalStackLayout Grid.Row="1">
        <Button Text="Add New Item" Command="{Binding AddNewCommand}" />
        <Button Text="Delete Selected" Command="{Binding DeleteSelectedCommand}" />
        <Button Text="Share Selected" Command="{Binding ShareSelectedCommand}" />
    </HorizontalStackLayout>
</Grid>

```

Ha nincs idő pontosan ugyanilyen UI-ra, akkor is legyen lista, kép, név, leírás, mennyiség és checkbox. A legtöbb pont a működő lista és binding miatt jár.

6. Gombstílus

A MainPage összes gombjára egyszerű lokális style:

```

<ContentPage.Resources>
    <Style TargetType="Button">
        <Setter Property="BackgroundColor" Value="MidnightBlue" />
        <Setter Property="FontAttributes" Value="Bold" />
        <Setter Property="Margin" Value="5" />
    </Style>
</ContentPage.Resources>

```

Ezt könnyű elfelejteni, pedig 5 gyors pont.

7. MainPage code-behind

A MainPage kapja meg DI-ból a ViewModelt, beállítja BindingContextnek, és itt történik a mentés/betöltés hívása.

```

public partial class MainPage : ContentPage
{
    MainPageViewModel viewModel;

    public MainPage(MainPageViewModel viewModel)
    {
        InitializeComponent();
        this.viewModel = viewModel;
        BindingContext = viewModel;
    }

    protected override async void OnAppearing()
    {
        base.OnAppearing();
        await viewModel.LoadFromJson();
    }

    protected override async void OnDisappearing()
    {
        base.OnDisappearing();
        await viewModel.SaveToJson();
    }
}

```

ZH-n ez nagyon fontos, mert a feladat konkrétan kéri az `OnDisappearing` mentést.

8. Új elem oldal

Hozz létre egy `NewItemPage.xaml` és `NewItemPage.xaml.cs` oldalt.

XAML-ben legyen:

- Entry névhez
- Entry leíráshoz
- Entry mennyiséghez
- képválasztó gomb
- Image előnézet
- Save gomb

Váz:

```

<VerticalStackLayout>
    <Label Text="Name" />
    <Entry Text="{Binding Name}" />
    <Label Text="Description" />
    <Entry Text="{Binding Description}" />
    <Label Text="Quantity" />
    <Entry Text="{Binding Quantity}" />
    <Button Text="Select image" Clicked="SelectImage_Clicked" />
    <Image Source="{Binding ImageURL}" WidthRequest="200" />
    <Button Text="Save" Clicked="Save_Clicked" />
</VerticalStackLayout>

```

Code-behind:

```

public partial class NewItemPage : ContentPage
{
    public ShoppingListItem NewItem { get; set; }

    public NewItemPage()
    {
        InitializeComponent();
        NewItem = new ShoppingListItem();
        BindingContext = NewItem;
    }
}

```

9. Új elem visszaküldése a főoldalra

Save gomb:

```
private async void Save_Clicked(object sender, EventArgs e)
{
    var param = new ShellNavigationQueryParameters()
    {
        { "newitem", NewItem }
    };

    await Shell.Current.GoToAsync("..", param);
}
```

Kritikus pont: a kulcs neve pontosan egyezzen a ViewModelben lévő QueryProperty kulccsal:

```
[QueryProperty(nameof(NewItem), "newitem")]
```

Ha itt előrod a `newitem` szót, az új elem nem fog megjelenni a listában.

10. Kép választása

MediaPicker:

```
private async void SelectImage_Clicked(object sender, EventArgs e)
{
    FileResult? photo = await MediaPicker.Default.PickPhotoAsync();

    if (photo != null)
    {
        string localFilePath = Path.Combine(FileSystem.Current.AppDataDirectory, photo.FileName);

        if (!File.Exists(localFilePath))
        {
            using Stream sourceStream = await photo.OpenReadAsync();
            using FileStream localFileStream = File.OpenWrite(localFilePath);
            await sourceStream.CopyToAsync(localFileStream);
        }

        NewItem.ImageURL = localFilePath;
    }
}
```

Fontos: ne csak az eredeti képre mutató hivatkozást tárold, hanem másold be az AppDataDirectory-ba. Így később is elérhető marad.

11. Törlés

ViewModelben:

```
[RelayCommand]
void DeleteSelected()
{
    if (SelectedItem != null)
    {
        ShoppingList.Remove(SelectedItem);
        SelectedItem = null;
    }
}
```

Ha nincs kiválasztva semmi, akkor vagy nem csinál semmit, vagy dob egy alertet/messengert. ZH-n már az is sokat ér, ha nem omlik össze.

12. Megosztás

Egyszerű megosztás:

```
[RelayCommand]
async Task ShareSelected()
{
    if (SelectedItem != null)
    {
        await Share.Default.RequestAsync(new ShareTextRequest()
        {
            Title = "Shopping Item",
            Text = $"Buy {SelectedItem.Quantity} {SelectedItem.Name}(s)!"
        });
    }
}
```

Ez csak 5 pont, ezért ne tölts vele túl sok időt. Elég, ha valamilyen módon megoszthatók a termékadatok.

13. JSON mentés és betöltés

ViewModelben:

```
string path = Path.Combine(FileSystem.Current.AppDataDirectory, "shoppingdata.json");
```

Betöltés:

```
public async Task LoadFromJson()
{
    if (File.Exists(path) && ShoppingList.Count == 0)
    {
        string jsonstring = await File.ReadAllTextAsync(path);
        List<ShoppingListItem> jsonData =
            JsonSerializer.Deserialize<List<ShoppingListItem>>(jsonstring);

        jsonData.ForEach(item => ShoppingList.Add(item));
    }
}
```

Mentés:

```
public async Task SaveToJson()
{
    string jsonstring = JsonSerializer.Serialize(ShoppingList);
    await File.WriteAllTextAsync(path, jsonstring);
}
```

Kritikus pontok:

- kell `using System.Text.Json;`
- `ObservableCollection` menthető JSON-ba
- betöltésnél ne add hozzá újra és újra ugyanazokat, ezért van a `ShoppingList.Count == 0`
- `AppDataDirectory` kell, nem tetszőleges Windows path

14. Shell és About oldal

Az `AppShell.xaml` legyen Flyoutos.

```

<Shell
    x:Class="MAUIMintaZH.AppShell"
    xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:local="clr-namespace:MAUIMintaZH"
    Shell.FlyoutBehavior="Flyout">

    <ShellContent
        Title="Shopping List"
        ContentTemplate="{DataTemplate local:MainPage}"
        Route="MainPage" />

    <ShellContent
        Title="About"
        ContentTemplate="{DataTemplate local:AboutPage}"
        Route="About" />
</Shell>

```

Az AboutPage.xaml minimum:

```

<VerticalStackLayout>
    <Label Text="This is the best test!"
        HorizontalOptions="Center"
        VerticalOptions="Center" />
</VerticalStackLayout>

```

Figyelj: a PDF-ben a szöveg `This is the best test!`. A tanári megoldásban nagybetűs változat szerepel, de éles ZH-n inkább a feladat szövegét kövesd pontosan.

15. Route regisztráció

Az új elem oldal route-ját regisztrálni kell az AppShell.xaml.cs fájlban:

```

public partial class AppShell : Shell
{
    public AppShell()
    {
        InitializeComponent();
        Routing.RegisterRoute("newitem", typeof(NewItemPage));
    }
}

```

A ViewModelben az új oldalra navigálás:

```

[RelayCommand]
async Task AddNewAsync()
{
    await Shell.Current.GoToAsync("newitem");
}

```

16. DI regisztráció

A MauiProgram.cs fájlban:

```

builder.Services.AddSingleton<MainPageViewModel>();
builder.Services.AddSingleton<MainPage>();

```

A tanári megoldásban a `NewItemPage` nincs DI-ba regisztrálva, mert paraméter nélküli konstruktora van és route alapján nyílik.

Ha saját ViewModeles új oldalt csinálnál, akkor azt is regisztrálni kellene, de ZH-n az egyszerű code-behinded új oldal gyorsabb.

Kritikus buktatók

- Ne felejtse el a `BindingContext = viewModel;` sort a `MainPage`-ben.
- Ne felejtse el a `Routing.RegisterRoute("newitem", typeof(NewItemPage));` sort.

- A `GoToAsync("newitem")` route neve egyezzen a regisztrált route-tal.
- A visszaadott paraméter kulcsa egyezzen: `"newitem"`.
- A `QueryProperty` a `ViewModel` osztályon legyen, ne véletlenül a modellen.
- `ObservableCollection` legyen a lista, ne sima `List`, mert a UI így frissül automatikusan.
- A modell legyen `partial` és örököljön `ObservableObject`-ből, ha `[ObservableProperty]` van benne.
- A `ViewModel` is legyen `partial`, ha `[ObservableProperty]` vagy `[RelayCommand]` van benne.
- A command neve XAML-ben a metódusnévből képződik: `AddNewAsync` -> `AddNewCommand`.
- A kép `ImageURL` property-je változzon, ne csak egy lokális változó.
- Mentésnél `FileSystem.Current.AppDataDirectory` legyen.
- Loadnál kezeld azt, ha még nem létezik fájl.
- Ha `Entry.Text="{Binding Quantity}"` intre bindol, rossz inputnál lehet gond, ezért ZH-n írd számot teszteléskor.
- Ha a képválasztás nem működik emulátoron, ne pánikolj: a fő pontok nem ezen múlnak.

Időbeosztás ZH-n

Ajánlott sorrend időnyomásban:

1. Modell és ViewModel lista: 10 perc
2. MainPage CollectionView és gombok: 20 perc
3. Shell route és NewItemPage: 20 perc
4. Visszanavigálás paraméterrel: 10 perc
5. Törlés és megosztás: 10 perc
6. JSON mentés/betöltés: 15 perc
7. About Flyout: 10 perc
8. Gyors teszt és hibajavítás: maradék idő

Ha elakadsz, a sorrend legyen: lista működjön, új elem hozzáadás működjön, törlés működjön, mentés működjön. A design és extra finomítás csak ezután.

Mit írnék az 1 A4-es kézzel írt jegyzetre?

Egy A4 oldalra nem teljes kódot érdemes írni, hanem „kulcsmintákat”. Ezek azok a részek, amik ZH-n könnyen kicsúsznak a fejből.

1. Modell minta

```
public partial class Item : ObservableObject {
    [ObservableProperty] string name;
    [ObservableProperty] string description;
    [ObservableProperty] int quantity;
    [ObservableProperty] string imageURL;
    [ObservableProperty] bool isDone;
}
```

2. ViewModel váz

```
[QueryProperty(nameof(NewItem), "newitem")]
public partial class MainVM : ObservableObject {
    public ObservableCollection<Item> Items { get; } = new();
    [ObservableProperty] Item selectedItem;
    public Item NewItem { set => Items.Add(value); }

    [RelayCommand] async Task AddNewAsync()
        => await Shell.Current.GoToAsync("newitem");

    [RelayCommand] void DeleteSelected() {
        if (SelectedItem != null) { Items.Remove(SelectedItem); SelectedItem = null; }
    }
}
```

3. Shell route + DI

```
Routing.RegisterRoute("newitem", typeof(NewItemPage));
builder.Services.AddSingleton<MainPageViewModel>();
builder.Services.AddSingleton<MainPage>();
```

4. Visszanavigálás új elemmel

```
var p = new ShellNavigationQueryParameters { { "newitem", NewItem } };
await Shell.Current.GoToAsync("..", p);
```

5. CollectionView binding váz

```
<CollectionView ItemsSource="{Binding Items}"
SelectedItem="{Binding SelectedItem}" SelectionMode="Single">
<CollectionView.ItemTemplate>
<DataTemplate x:DataType="local:Item">
<Grid ColumnDefinitions="*,*,3*">
<CheckBox IsChecked="{Binding IsDone}" />
<Image Source="{Binding ImageURL}" Grid.Column="1" />
<VerticalStackLayout Grid.Column="2">
<Label Text="{Binding Name}" />
<Label Text="{Binding Description}" />
<Label Text="{Binding Quantity}" />
</VerticalStackLayout>
</Grid>
</DataTemplate>
</CollectionView.ItemTemplate>
</CollectionView>
```

6. JSON mentés/betöltés

```
string path = Path.Combine(FileSystem.Current.AppDataDirectory, "data.json");
await File.WriteAllTextAsync(path, JsonSerializer.Serialize(Items));
if (File.Exists(path) && Items.Count == 0)
foreach (var i in JsonSerializer.Deserialize<List<Item>>(await File.ReadAllTextAsync(path)))
    Items.Add(i);
```

7. Képválasztás

```
var photo = await MediaPicker.Default.PickPhotoAsync();
if (photo != null) {
var path = Path.Combine(FileSystem.Current.AppDataDirectory, photo.FileName);
using var s = await photo.OpenReadAsync();
using var f = File.OpenWrite(path);
await s.CopyToAsync(f);
NewItem.ImageURL = path;
}
```

8. Share

```
await Share.Default.RequestAsync(new ShareTextRequest {
Title = "Item",
Text = $"{SelectedItem.Quantity} {SelectedItem.Name}"
});
```

9. MainPage életciklus

```
protected override async void OnAppearing() { await vm.LoadFromJson(); }
protected override async void OnDisappearing() { await vm.SaveToJson(); }
```

10. XAML gomb style

```
<Style TargetType="Button">
  <Setter Property="BackgroundColor" Value="MidnightBlue"/>
  <Setter Property="FontAttributes" Value="Bold"/>
  <Setter Property="Margin" Value="5"/>
</Style>
```

Minimum működő megoldás stratégia

Ha kevés az idő, ezt csináld:

1. Modell legyen meg.
2. MainPage-en jelenjen meg egy lista.
3. Új oldalon lehessen adatot beírni.
4. Save után kerüljön vissza a főoldalra.
5. Lehessen törölni kiválasztott elemet.
6. Legyen JSON mentés/betöltés.
7. Legyen About oldal.

Ha ezek működnek, már a pontok nagy részét el lehet hozni. A képválasztás, megosztás és pontos design fontos, de kisebb súlyú.

Gyors önellenőrző lista beadás előtt

- Elindul az app?
- A főoldalon látszik a lista?
- Az Add New Item gomb új oldalra visz?
- A Save visszahoz a főoldalra?
- Az új elem megjelenik a listában?
- Kiválasztás után törölhető az elem?
- A megosztás gomb nem dob hibát?
- Az About oldal elérhető a Flyoutból?
- Bezárás/navigálás után visszatöltődik a lista?
- A gombok MidnightBlue színűek és félkövérek?

Mentális modell

Ha fejben össze akarod rakni:

```
ShoppingListItem = egy termék adatai
MainPageViewModel = lista + commandok + JSON
MainPage.xaml = lista megjelenítése + gombok
NewItemPage = új ShoppingListItem kitöltése
Shell route = oldalak közti navigáció
QueryProperty = új elem visszajuttatása MainPageViewModelbe
OnDisappearing/OnAppearing = mentés/betöltés
```

Ezt a sorrendet érdemes begyakorolni, mert az éles feladat valószínűleg csak témában lesz más. Például bevásárlólista helyett könyvlista, filmek, feladatlista, receptlista vagy jegyzetlista, de a szerkezet ugyanaz marad.